

CS 303

FRONTEND FRAMEWORKS

ANNOUNCEMENTS

1. Next week = Guest Speaker week!
 - a. Mon: **[No in-person class]** watch guest speaker recording
 - b. Wed: Sr. Software Eng @ PayPal
2. Feedback deadline next Weds for Project #1
3. Team assignments for final project

4 COMMONALITIES

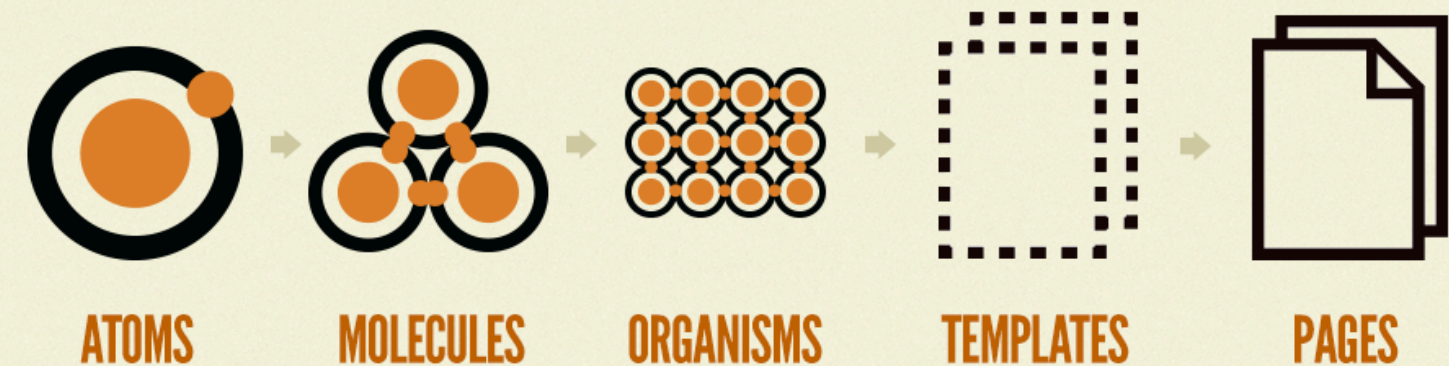
- 1.Components
- 2.State Handling
- 3.Special rendering (ex: VDOM)
- 4.Dependency Injection

COMPONENTS

Components Thinking:

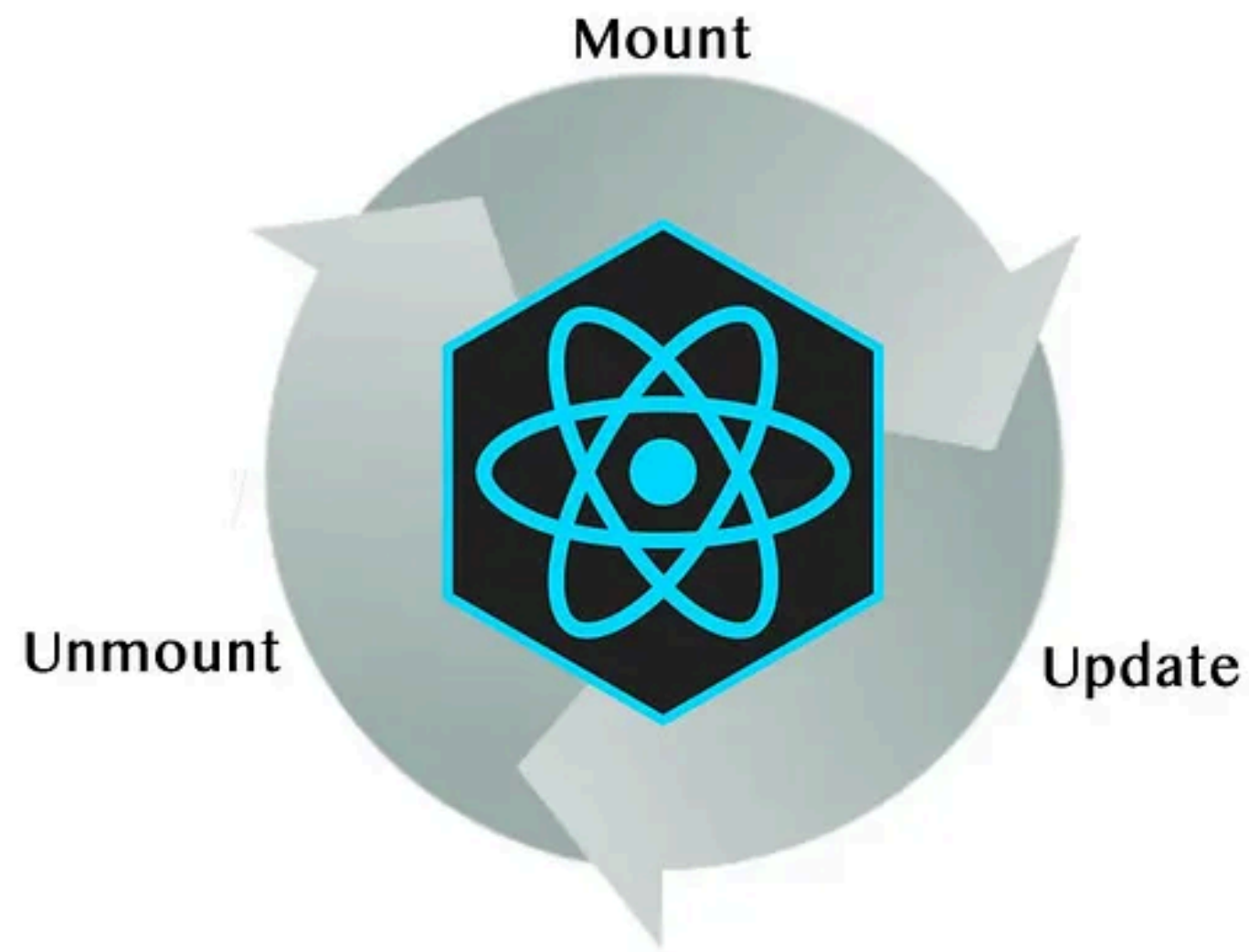
- Breaking down the UI into the smallest usable unit and then building from here

Composability: – components can be made from other components



STATE HANDLING IN THE FE

Component Lifecycle



Trigger render

2 reasons to do this

1. Component's initial render
2. Component's state (or ancestor) has been updated

Render component

Commit to DOM

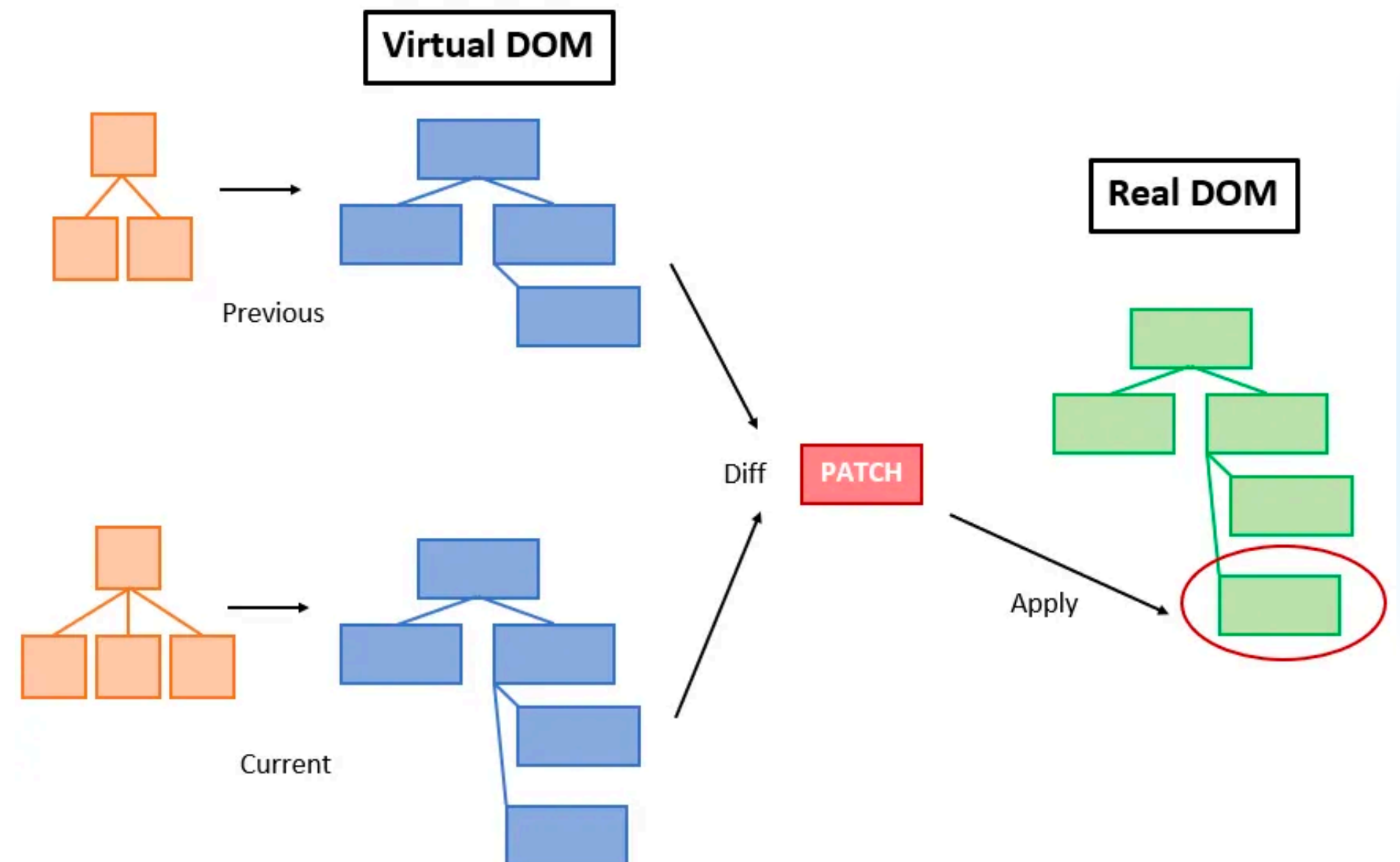
REFLECTION

For your learning doc:

1. In your own words – what is the Virtual DOM and why is it more efficient than manipulating the physical browser DOM?
2. What is a design pattern?
3. When would you leverage `useMemo` over `useEffect`?

VIRTUAL DOM (VDOM)

1. Construct initial virtual DOM
2. When changes are made:
 - a. Rerender a new virtual DOM tree
 - b. Diff against previous virtual DOM tree
3. Make changes to the actual (physical) DOM



4 COMMONALITIES

1. Components
2. State Handling
3. Special rendering (ex: VDOM)
4. **Dependency Injection**

WHY DESIGN PATTERNS?

01

Proven solutions

These are approaches that have been well tested by the developer community

03

Provides faster communication

You now have the same language to describe a problem and its subsequent solution

02

Reusable

Patterns are robust and can be easily adapted to fit needs

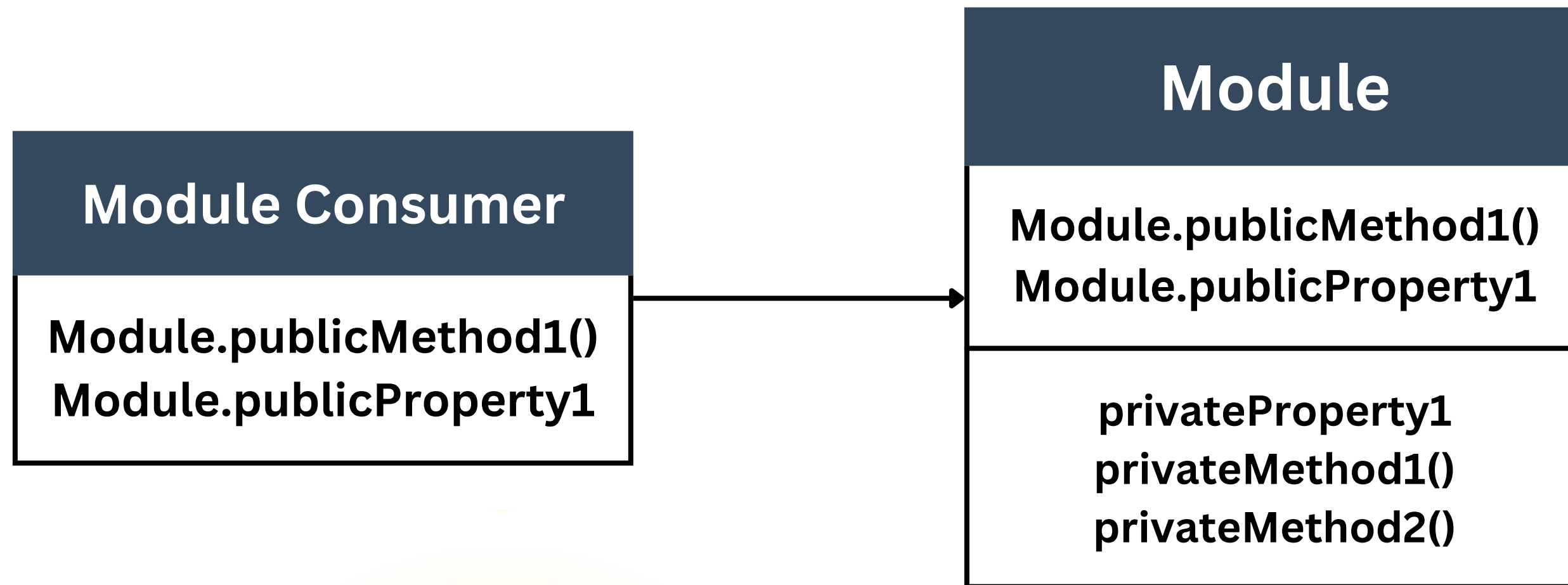
04

Makes code less spaghetti like

Software systems can have better structure

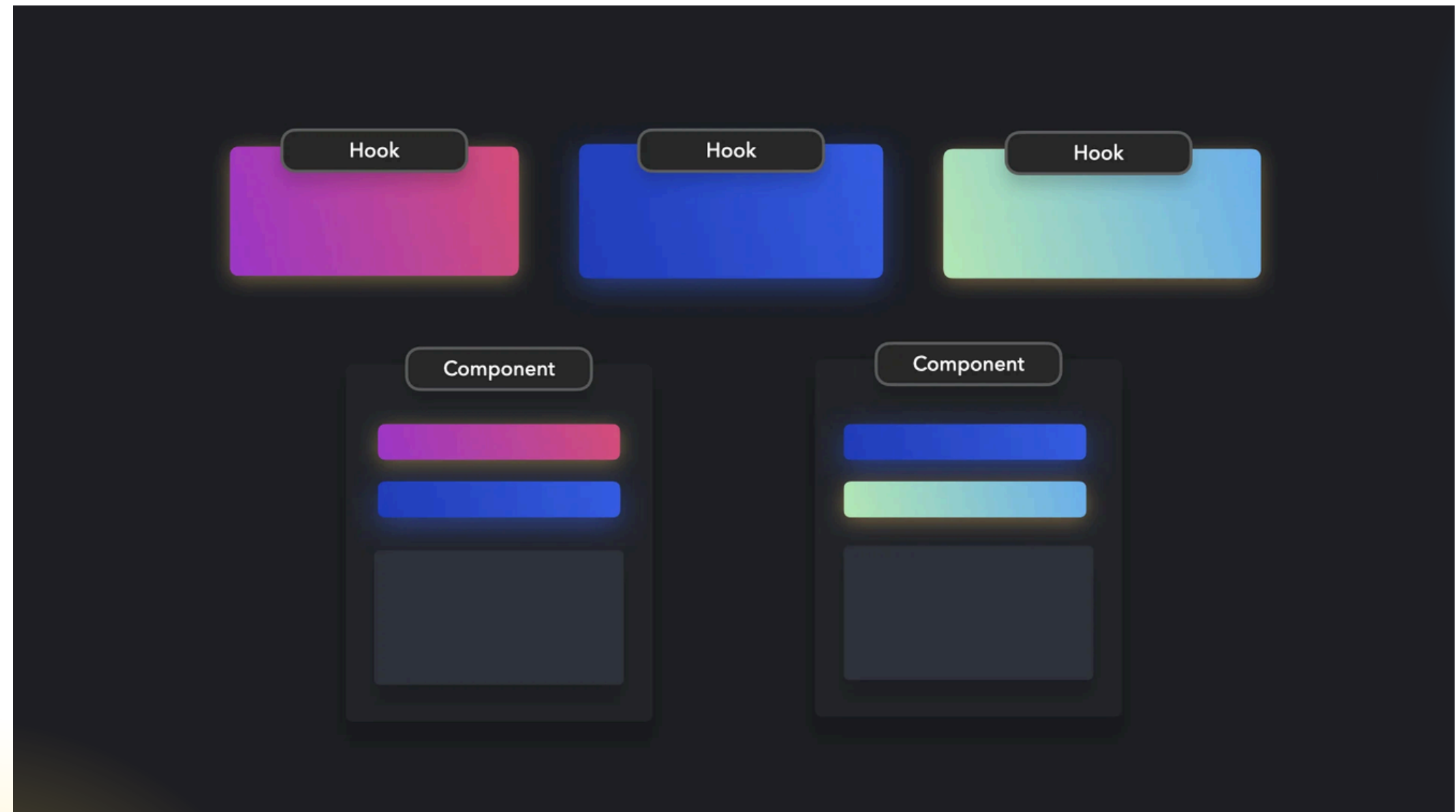
THE MODULE PATTERN

- Provides private/public encapsulation
- Allows us to use import/export
- Prevents our functions/variables from polluting the global namespace
- Closure -- things defined in the module stay in the module unless you define otherwise



THE HOOKS PATTERN

- React specific design pattern that allows us to dictate what is known as a **reactive effect** (side effect at different points of the component lifecycle)
 - We've seen a few of these already — useState (for instance)
- Other hooks:
 - useEffect
 - useLayoutEffect
 - useMemo
 - useContext
 - useReducer
 -many more
 - And custom hooks



REACTIVE EFFECTS

```
const todos = useSomeDataLibrary(`/api/user/${userId}/todos`);
```

useEffect: Effects can start/stop synchronization

- Goal is to sync external items (ex: data fetching) to your components current prop/state
- Runs after every render by default
 - Can set dependencies (deps) to re-run only when the deps change
 - Might need cleanup (the code that should run when the component unmounts)

```
useEffect(() => {  
  let ignore = false;  
  
  async function startFetching() {  
    const json = await fetchTodos(userId);  
    if (!ignore) {  
      setTodos(json);  
    }  
  }  
  
  startFetching();  
  
  return () => {  
    ignore = true;  
  };  
}, [userId]);
```

REACTIVE EFFECTS

useLayoutEffect – builds on top of `useEffect` but runs before the browser paint (rasterizes)

```
function Tooltip() {  
  const ref = useRef(null);  
  const [tooltipHeight, setTooltipHeight] = useState(0); // You don't know real height yet  
  
  useLayoutEffect(() => {  
    const { height } = ref.current.getBoundingClientRect();  
    setTooltipHeight(height); // Re-render now that you know the real height  
  }, []);  
  
  // ...use tooltipHeight in the rendering logic below...  
}
```


CRITICAL RENDERING PATH

1. Get the initial HTML
2. Parse the HTML into the DOM
3. Parse CSS into the CSSOM
4. Create render tree
5. Determine where render tree nodes should appear on the page (layout)
6. Paint nodes onto screen (rasterize)



REACTIVE EFFECTS

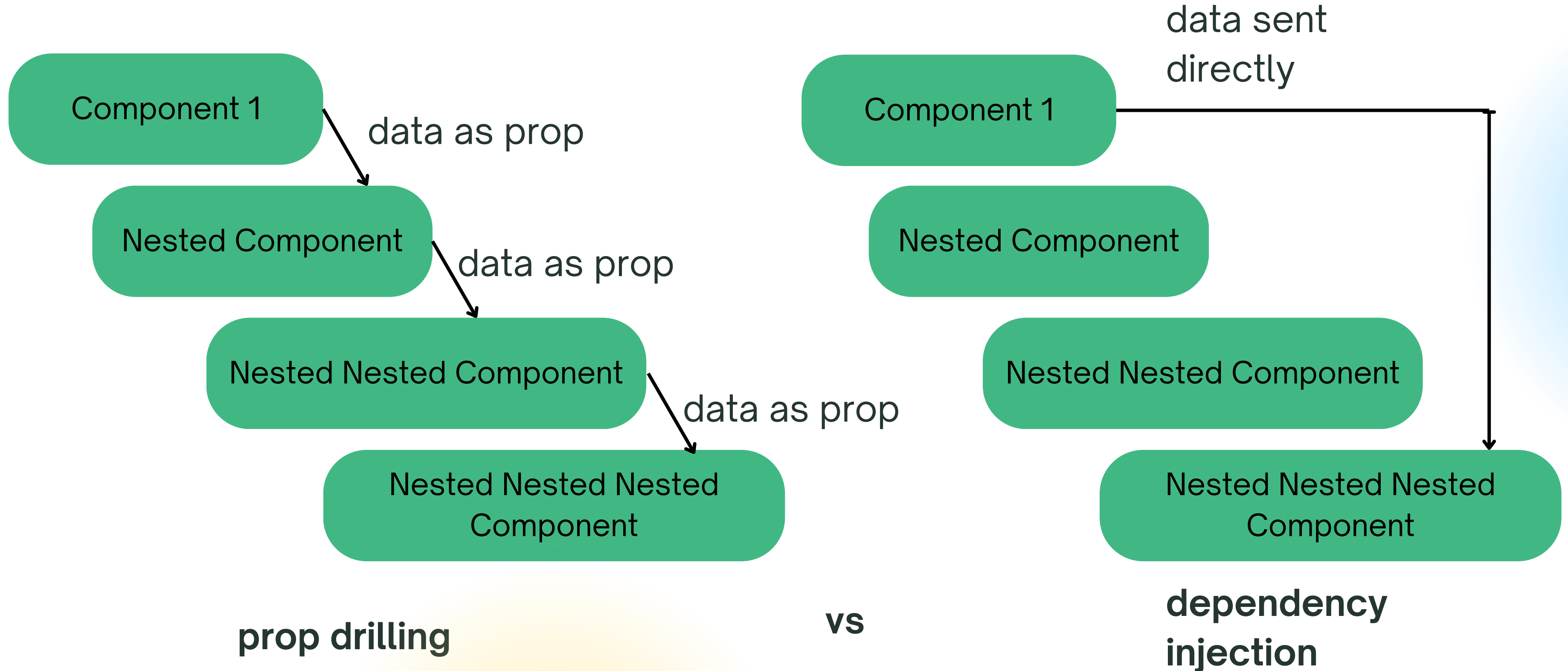
useMemo – for caching expensive calculations and only runs on initial render + when deps change

```
import { useMemo } from 'react';

function TodoList({ todos, tab }) {
  const visibleTodos = useMemo(
    () => filterTodos(todos, tab),
    [todos, tab]
  );
  // ...
}
```

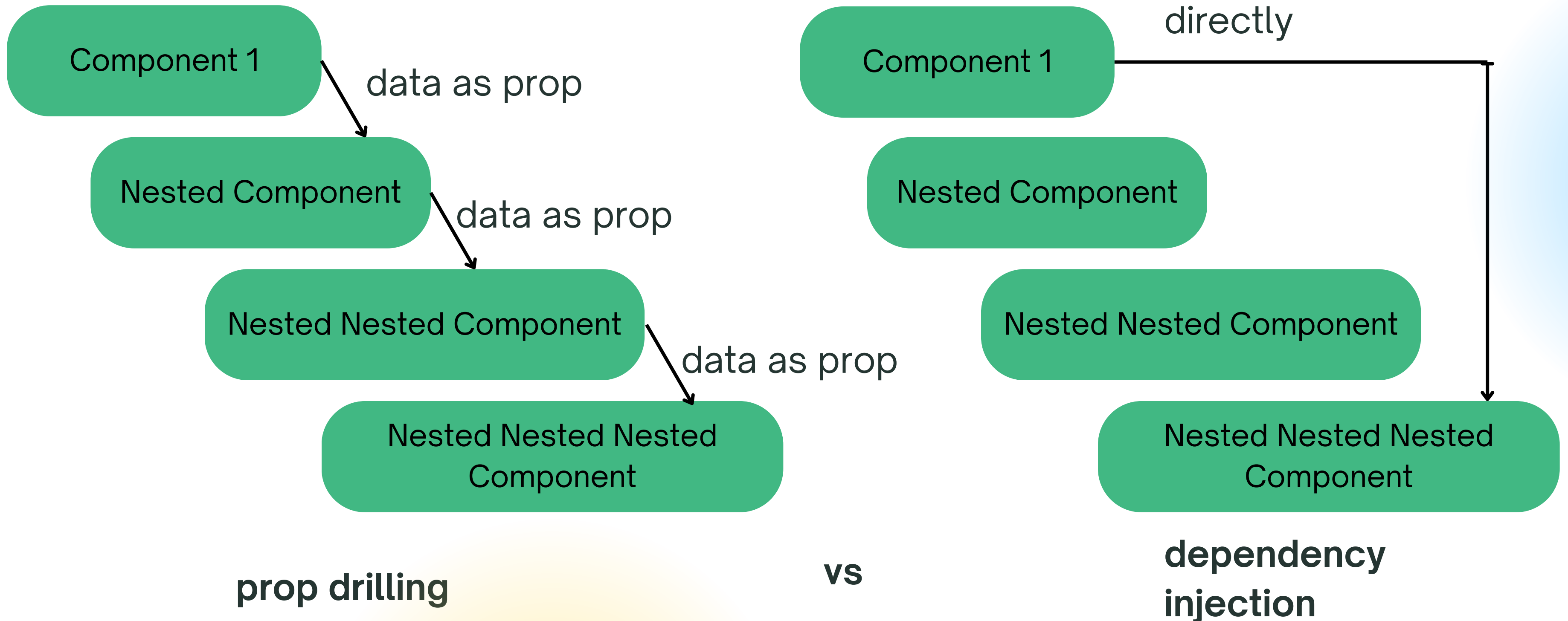

DEPENDENCY INJECTION

- A way to access data directly from even the most nested component without having to pass data down all the intermediary components



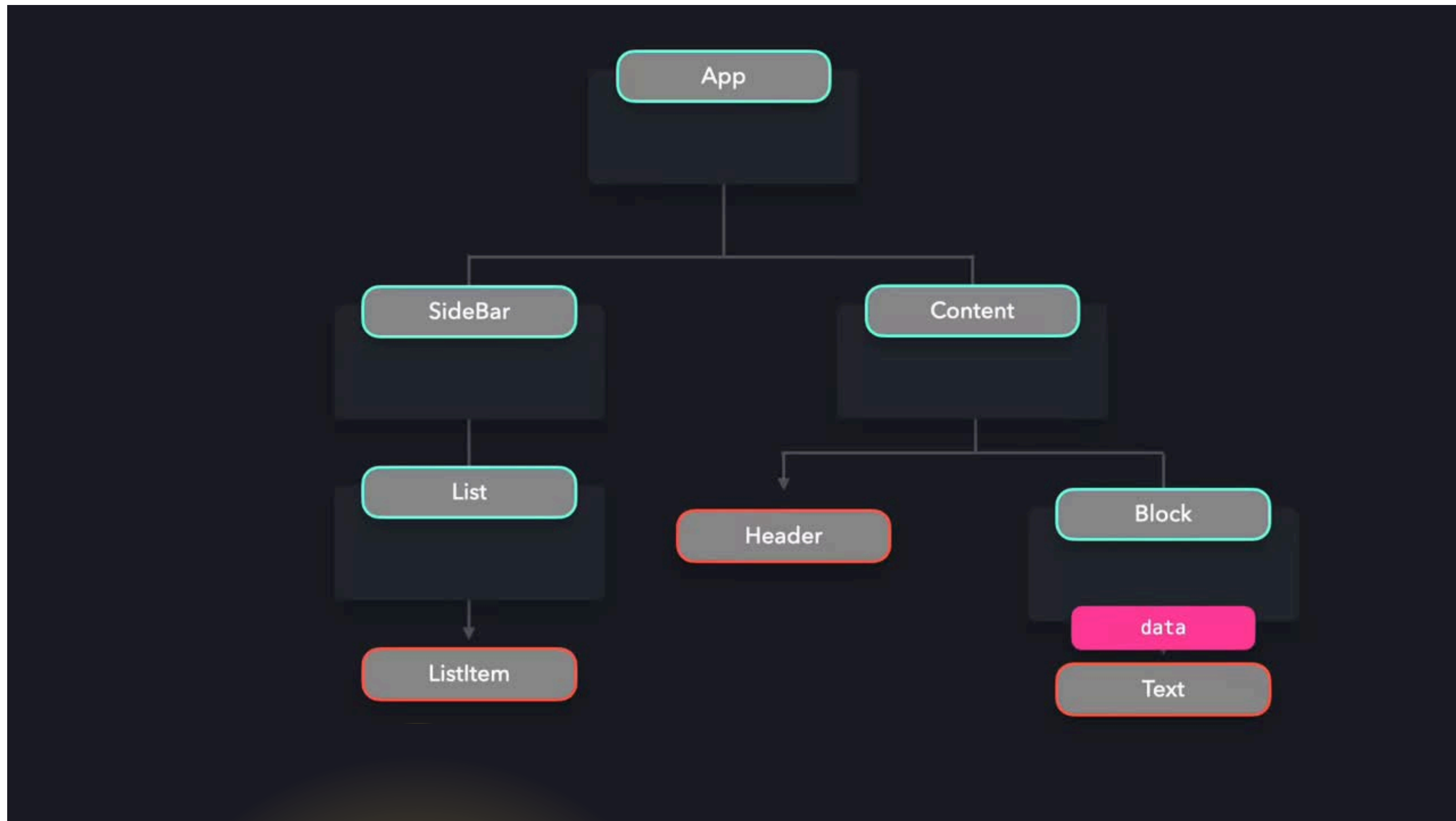
DEPENDENCY INJECTION

- A way to access data directly from even the most nested component without having to pass data down all the intermediary components
 - stackblitz.com/edit/cs303-react-todos-context data sent directly



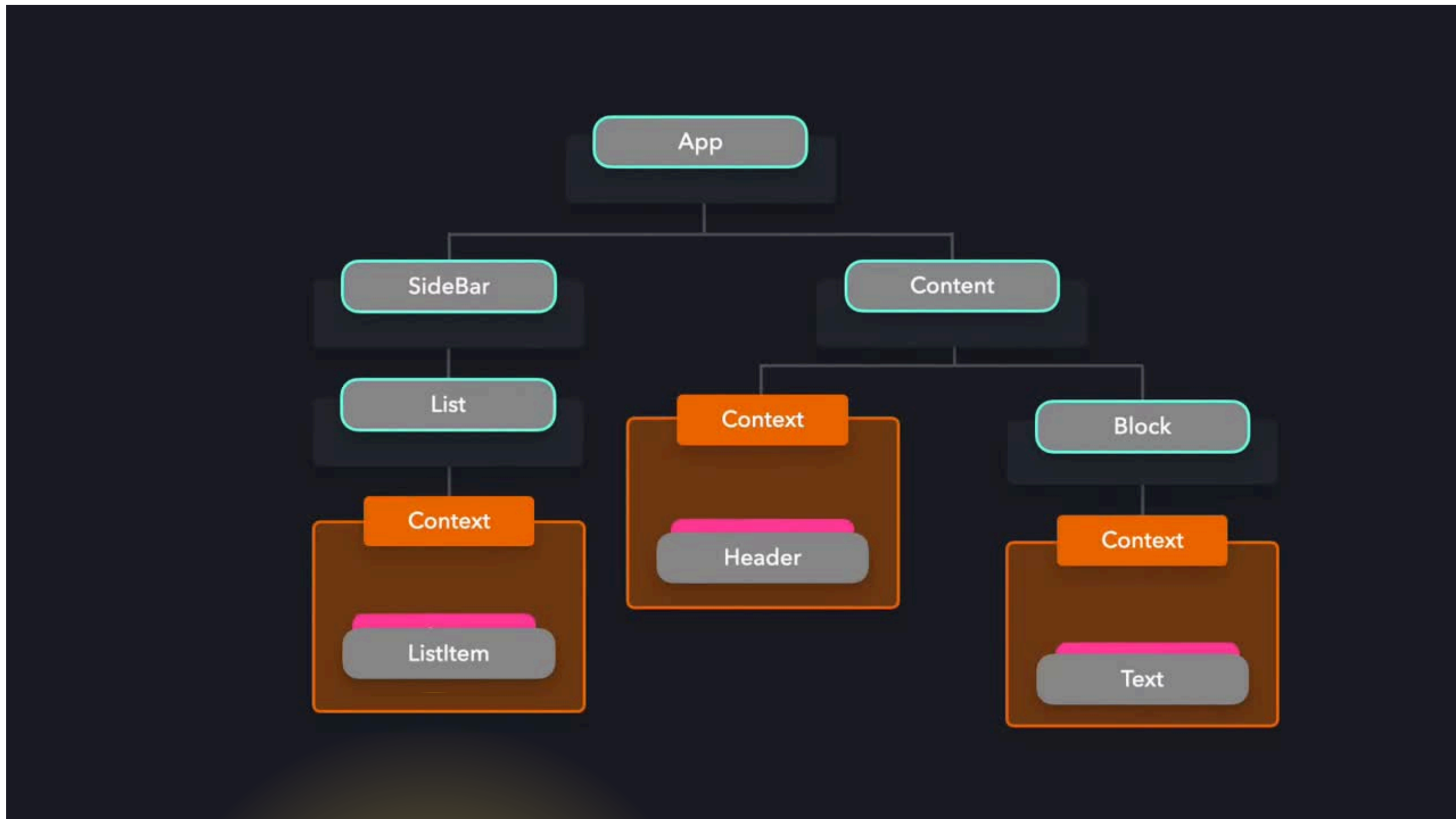
DEPENDENCY INJECTION (DI)

Prop drilling – passing data (prop) down many layers of the component tree



DEPENDENCY INJECTION (DI)

DI – only passing in the data to the components that need the data (context)



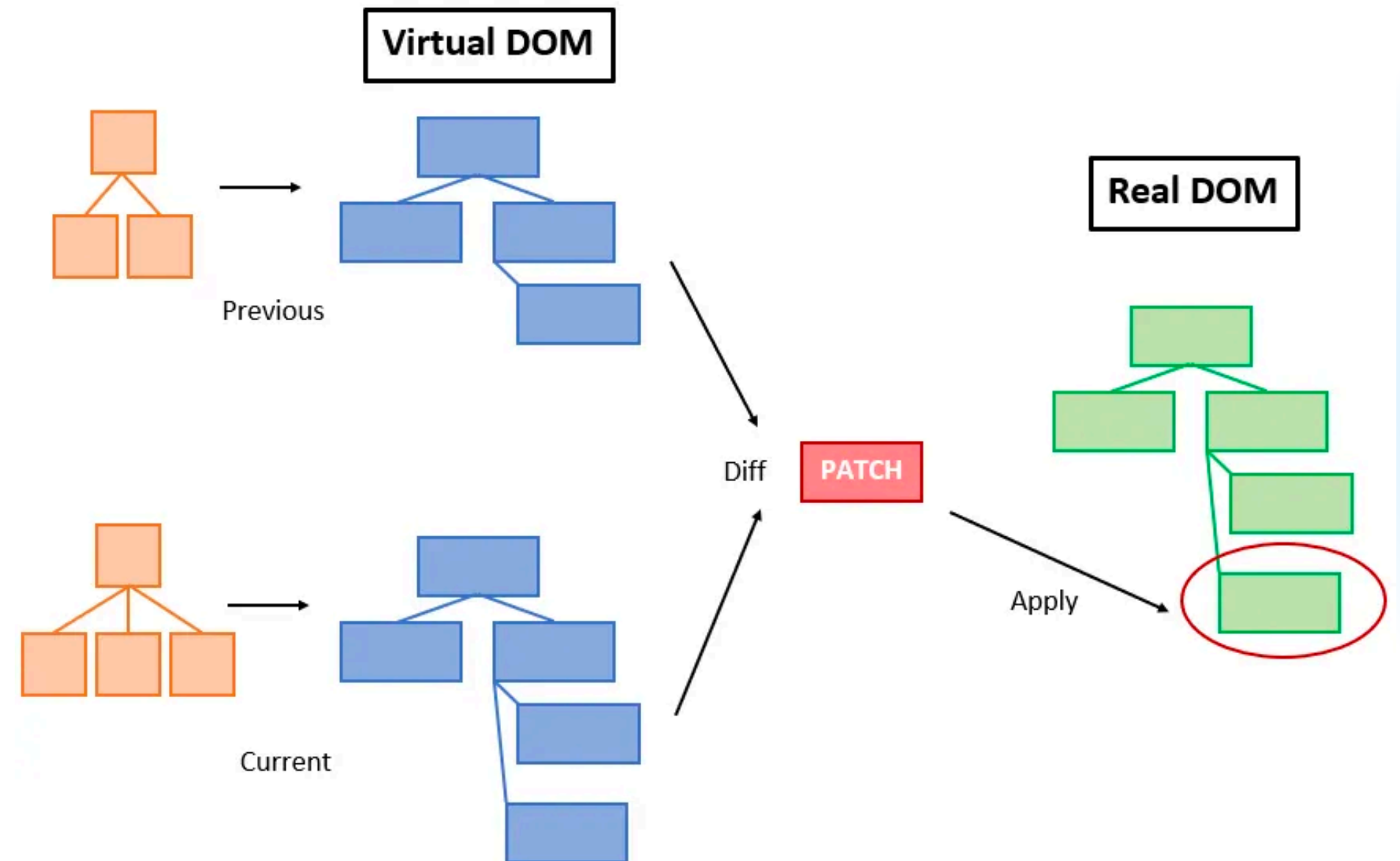
4 COMMONALITIES

- 1.Components
- 2.State Handling
- 3.Special rendering (ex: VDOM)
- 4.Dependency Injection

WHERE DOES NEXT.JS FIT IN?

VIRTUAL DOM (VDOM)

1. Construct initial virtual DOM
2. When changes are made:
 - a. Rerender a new virtual DOM tree
 - b. Diff against previous virtual DOM tree
3. Make changes to the actual (physical) DOM

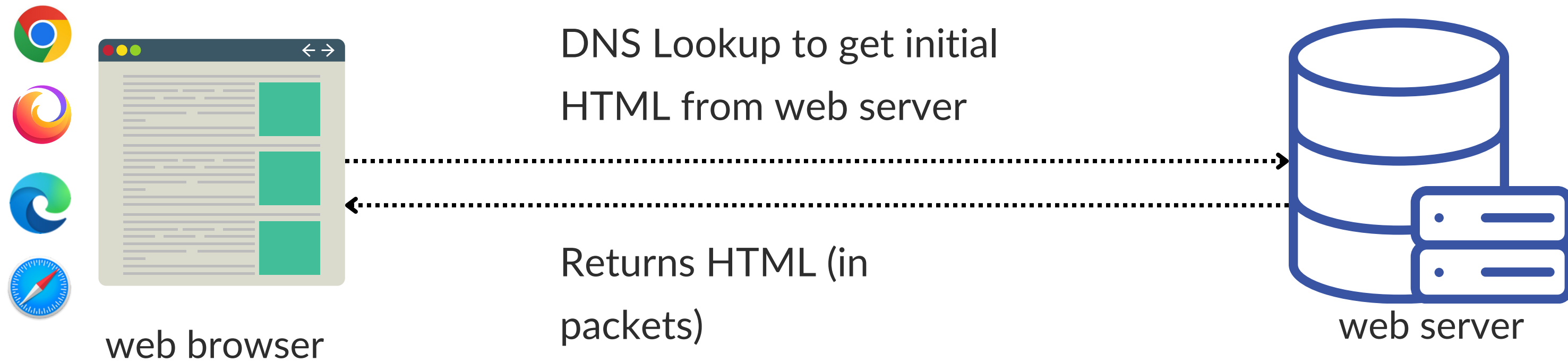


CRITICAL RENDERING PATH

1. Get the initial HTML 
2. Parse the HTML into the DOM
3. Parse CSS into the CSSOM
4. Create render tree
5. Determine where render tree nodes should appear on the page (layout)
6. Paint nodes onto screen (rasterize)

HOW BROWSERS WORK

1. Get the initial HTML



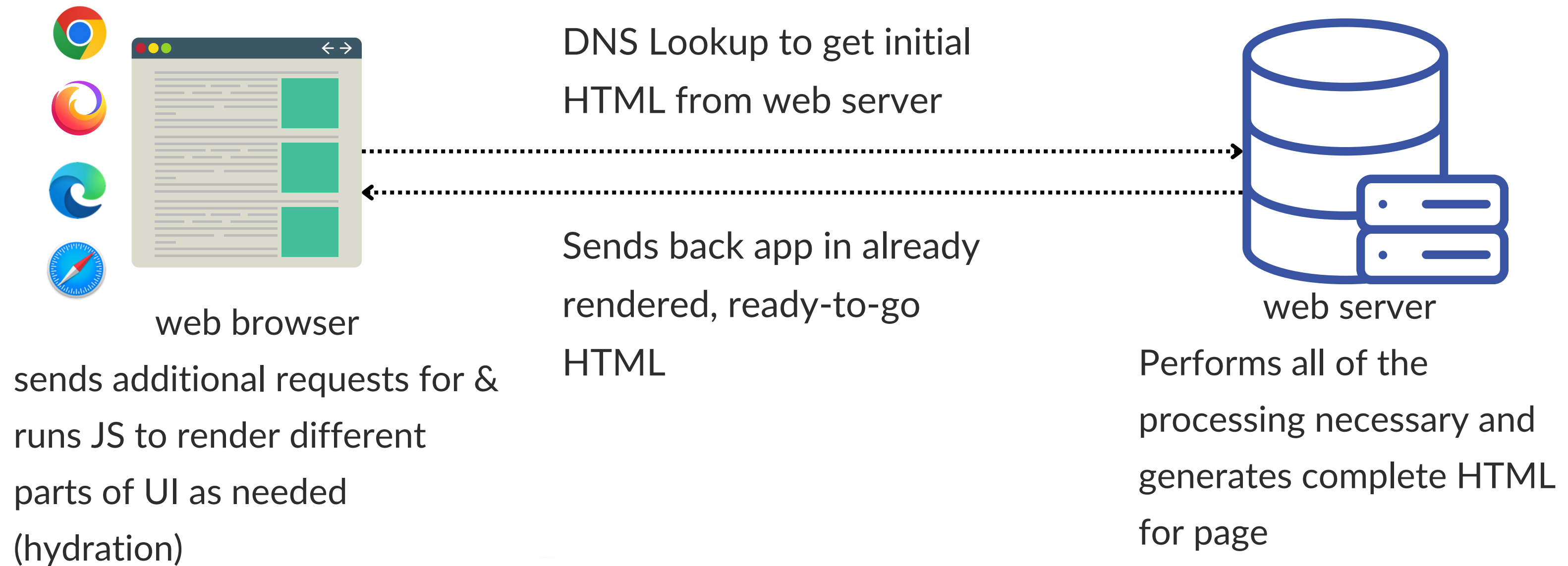
CSR

Client Side Rendering



SSR

Server Side Rendering



NEXT.JS

- Production grade React framework
 - Routing
 - SSR
 - CSR → ‘use client’
 - ex: when using some hooks
- App router

